

# Práctica de laboratorio: Quicksort en la práctica

## Análisis y Diseño de Algoritmos

### 1. Contexto

En las diapositivas del curso se presenta quicksort como un algoritmo de ordenamiento muy práctico, basado en dividir el arreglo alrededor de un pivote y ordenar recursivamente las dos partes. Además, se indica que quicksort suele ser más rápido que merge sort en la práctica, pero que su rendimiento depende de detalles de implementación. En esta práctica se usará una base de código para implementar quicksort, medir su comportamiento y comparar sus resultados con merge sort.

La práctica sigue el enfoque de estudio de caso propuesto por Ferri y Vidal: partir de un algoritmo conocido, instrumentarlo, obtener medidas empíricas y representar gráficamente los resultados para discutir el comportamiento observado. El énfasis no estará en una demostración matemática extensa, sino en la medición controlada, la repetibilidad y la interpretación técnica de los datos.

### 2. Duración

La práctica está diseñada para desarrollarse en una sesión de laboratorio de 1 hora y 30 minutos.

### 3. Objetivos

1. Implementar correctamente la partición de quicksort dentro de una base de código proporcionada.
2. Ejecutar un experimento pequeño y repetible para comparar quicksort y merge sort.
3. Verificar automáticamente que los arreglos quedan ordenados después de cada ejecución.
4. Registrar mediciones de tiempo en formato CSV.
5. Generar gráficos simples que permitan comparar los tiempos obtenidos.
6. Interpretar los resultados considerando el tamaño de entrada y el tipo de entrada.

### 4. Repositorio base

Se entregará un repositorio con la siguiente estructura:

```
quicksort-lab/  
  Makefile  
  README.md  
  include/  
  src/  
  scripts/
```

```
results/  
doc/
```

El código ya incluye generación de arreglos, medición de tiempo, verificación de ordenamiento, una implementación de merge sort y scripts básicos para ejecutar el experimento. La parte principal que debe completar es la implementación de quicksort en el archivo:

```
src/quicksort.c
```

## 5. Actividad principal

Complete la función de partición de quicksort. Puede usar como pivote el último elemento del subarreglo. La partición debe reordenar los elementos de modo que, al finalizar, los elementos menores o iguales que el pivote queden a su izquierda y los mayores queden a su derecha. La función debe devolver la posición final del pivote.

Una vez implementada la partición, compile el proyecto con:

```
make
```

Luego ejecute una prueba rápida:

```
make test
```

Toda línea producida por el programa debe terminar con `sorted = 1`. Si alguna ejecución produce `sorted = 0`, el algoritmo no está ordenando correctamente y los tiempos obtenidos no deben considerarse válidos.

## 6. Experimento

Ejecute el experimento pequeño incluido en el repositorio:

```
make run
```

El programa generará un archivo CSV en:

```
results/results.csv
```

El archivo tendrá el siguiente formato:

```
algorithm,inputType,n,seed,timeSeconds,sorted
```

Los tipos de entrada considerados son:

- a) **random**: arreglo aleatorio con elementos distintos.
- b) **sorted**: arreglo ya ordenado.
- c) **reversed**: arreglo inversamente ordenado.
- d) **repeated**: arreglo con muchos elementos repetidos.

## 7. Gráficos

Genere los gráficos usando:

`gnuplot scripts/plot.gp`

El script generará, como mínimo, los siguientes gráficos para la entrada aleatoria:

1. Tiempo promedio de quicksort y merge sort.
2. Tiempo normalizado por  $n \log_2 n$ .
3. Razón de tiempos:

$$R(n) = \frac{T_{\text{merge}}(n)}{T_{\text{quick}}(n)}.$$

Si  $R(n) > 1$ , quicksort fue más rápido en promedio para ese tamaño de entrada. Si  $R(n) < 1$ , merge sort fue más rápido.

## 8. Distribución sugerida del tiempo

1. 10 minutos: revisar la estructura del repositorio y compilar el proyecto.
2. 30 minutos: completar la partición de quicksort y verificar la corrección.
3. 20 minutos: ejecutar el experimento y revisar el archivo CSV.
4. 15 minutos: generar los gráficos.
5. 15 minutos: escribir una breve interpretación de los resultados.

## 9. Preguntas para el informe breve

Responda en un texto corto, de no más de una página:

1. ¿Quicksort ordenó correctamente todos los casos ejecutados?
2. Para entradas aleatorias, ¿qué algoritmo fue más rápido en promedio?
3. ¿La razón  $R(n)$  aumenta, disminuye o se mantiene relativamente estable cuando crece  $n$ ?
4. ¿Qué ocurrió con las entradas ordenadas e inversamente ordenadas?
5. ¿Qué ocurrió con las entradas con muchos elementos repetidos?
6. ¿Qué detalles de implementación podrían cambiar los resultados obtenidos?